Why Are They Called Hashes?

the individual keys are mapped to individual values, or *associative array* because they associate keys with values.

The name "Hash" (or the related Java term "HashMap") is named after an implementation detail. The keys are stored in memory based on a function that returns a (hopefully) unique value for each object. Because the location of each key can be found without referring to the entire object, lookup is very fast. The function that returns the unique value is called a *hashing function* and so the data structure is called a Hash.

The example that follows uses hash literals—a list of *key value* pairs between braces:

```
h = {"dog" => "canine", "cat" => "feline", "bear" => "ursine"}

h.length  # => 3
h["dog"]  # => "canine"
h["cow"] = "bovine"
h[12] = "dodecine"
h["cat"] = 99
h         # => {"dog"=>"canine", "cat"=>99, "bear"=>"ursine", "cow"=>"bovine",
          # .. 12=>"dodecine"}
```

In the previous example, the hash keys were strings, and the hash literal used => to separate the keys from the values. (The => is sometimes called a *hashrocket*.) If the keys are *symbols*, there is a shortcut you can use. You can still use => to separate symbol keys from values:

```
h = {:dog => "canine", :cat => "feline", :bear => "ursine"}
```

But you can also write the literal by moving the colon to the end of the symbol and dropping the =>.

```
h = { dog: "canine", cat: "feline", bear: "ursine"}
```

Because the value of a symbol doesn't change, symbols are often used as hash keys, and this shortcut is therefore very common.

There is an even shorter shortcut. Often when creating a new hash, you are using existing data stored in variables that share the same name as the key that the variable will be indexed under in the hash. Something like this:

```
firstname = "Fred"
lastname = "Flintstone"
user = {firstname: firstname, lastname: lastname}
puts user
```

produces:

```
{:firstname=>"Fred", :lastname=>"Flintstone"}
```

You don't need to duplicate the key and the value if they have the same name:

```
firstname = "Fred"
lastname = "Flintstone"
user = {firstname:, lastname:}
puts user
```